ONLINE BOUTIQUE ON AMAZON EKS

Sổ tay vận hành

Cách chạy, kiểm tra, thay đổi và tắt hệ thống

Mã tài liệu	OBE-RUN-001-VI
Phiên bản	2.1
Ngày	2026-10-10
Trạng thái	Đã duyệt cho môi trường dev
Người phụ trách	khaipd18 (DevOps / Cloud)
Đối tượng đọc	Kỹ sư triển khai hoặc vận hành hệ thống
Phân loại	Nội bộ
Repository	https://github.com/khaipd18/online-boutique-gitops-pipeline

Kiểm soát tài liệu

Lịch sử thay đổi

Phiên bản	Ngày	Thay đổi	Người thực hiện
1.0	2026-10-09	Phát hành lần đầu.	khaipd18
1.1	2026-10-10	Chạy thật trên tài khoản mới; Terraform quản lý log group của control plane.	khaipd18
1.2	2026-10-10	Thêm ảnh chụp từ lần triển khai đầy đủ trên EKS; thêm IR-15.	khaipd18
2.0	2026-10-10	Viết lại cho ngắn gọn: các việc chính theo từng bước đơn giản, mọi sự cố gộp vào một bảng, phần nâng cao chuyển xuống phụ lục.	khaipd18
2.1	2026-10-10	CI ghi tag image qua pull request; không còn phải tạm tắt quy tắc bảo vệ nhánh.	khaipd18

Tài liệu liên quan

- OBE-TDD-001-VI Tài liệu thiết kế kỹ thuật: vì sao hệ thống được xây như vậy
- Bản tiếng Anh: `docs/manuals/operations-runbook.en.pdf` (OBE-RUN-001)
- README.vi.md: tóm tắt dự án và cách chạy thử ở máy local với kind

Mục lục

1 Bắt đầu từ đây 3
1.1 Cái gì chạy ở đâu 3
1.2 Ba quy tắc 3
1.3 Công cụ cần có 3
2 Các việc chính 4
2.1 Dựng môi trường 4
2.2 Kiểm tra hệ thống chạy tốt 4
2.3 Ra bản mới cho một service 7
2.4 Rollback một service 7
2.5 Đổi cấu hình hoặc scale một service 8
2.6 Thay đổi hạ tầng 8
2.7 Tắt môi trường 8
3 Khi có sự cố 10
Phụ lục 12
A. Nâng phiên bản Kubernetes 12
B. Lỗ hổng bảo mật 12
C. Chi phí 13
D. Khôi phục 13
E. Service và port 13
F. Phiên bản 14

1 Bắt đầu từ đây

1.1 Cái gì chạy ở đâu

Thành phần	Nằm ở đâu
Hạ tầng (VPC, EKS, ECR, IAM)	AWS `ap-southeast-1`, do Terraform tạo từ thư mục `terraform/`
Cluster Kubernetes	`khaipd18-eks-cluster`, 3 node `t3.medium`
Shop (12 service)	Namespace `dev-eks`, Argo CD deploy từ `gitops/dev-eks/`
Argo CD	Namespace `argocd`, chạy bên trong cluster
Monitoring (Prometheus, Grafana)	Namespace `monitoring`
CI/CD	GitHub Actions trong repository

1.2 Ba quy tắc

1. **Mọi thay đổi đi qua Git.** Argo CD luôn đưa cluster về đúng như trong Git, nên sửa tay bằng `kubectl` sẽ bị hoàn lại sau vài phút.
2. **Thay đổi hạ tầng phải qua pull request.** Bước Terraform apply sẽ chờ bạn bấm duyệt trên GitHub.
3. **Dùng xong thì tắt môi trường.** Chạy tốn khoảng 0,40 USD mỗi giờ, chưa tính load balancer (Phụ lục C).

1.3 Công cụ cần có

AWS CLI v2, Terraform 1.14 trở lên, `kubectl`, Helm, GitHub CLI (`gh`). Đăng nhập AWS bằng profile của tài khoản rồi kiểm tra:

```
aws sts get-caller-identity          # xem đang dùng tài khoản nào
aws eks update-kubeconfig --region ap-southeast-1 --name khaipd18-eks-cluster
kubectl get nodes                    # 3 node, đều Ready
```

> **GHI CHÚ**
> Chỉ danh tính đã tạo cluster mới có quyền admin. Role dùng trên AWS console được Terraform tự cấp quyền chỉ đọc, nên EKS console xem được pod và node.

2 Các việc chính

2.1 Dựng môi trường

Thời gian: khoảng 45 phút. **Bắt đầu tính tiền từ bước 3.**

1. **Báo cho GitHub biết dùng tài khoản AWS nào:**

```
gh variable set AWS_ACCOUNT_ID --body <account-id>
```

2. **Tạo chỗ lưu state của Terraform** (mỗi tài khoản chỉ làm một lần):

```
aws s3api create-bucket --bucket <state-bucket> --region ap-southeast-1 \
  --create-bucket-configuration LocationConstraint=ap-southeast-1
aws s3api put-bucket-versioning --bucket <state-bucket> \
  --versioning-configuration Status=Enabled
aws dynamodb create-table --table-name <lock-table> --region ap-southeast-1 \
  --attribute-definitions AttributeName=LockID,AttributeType=S \
  --key-schema AttributeName=LockID,KeyType=HASH --billing-mode PAY_PER_REQUEST
```

   Tên phải khớp với `terraform/backend.tf`.

3. **Tạo hạ tầng** (khoảng 17 phút):

```
cd terraform
terraform init
terraform plan -out tfplan     # đọc plan: 74 resource sẽ được tạo
terraform apply tfplan
```

4. **Cài Argo CD và chỉ cho nó những gì cần deploy:**

```
aws eks update-kubeconfig --region ap-southeast-1 --name khaipd18-eks-cluster
kubectl create namespace argocd
kubectl apply -n argocd --server-side -f https://raw.githubusercontent.com/argoproj/
argo-cd/stable/manifests/install.yaml
kubectl apply -f gitops/argocd/namespaces.yaml
kubectl apply -f gitops/argocd/applicationset.yaml
kubectl apply -f gitops/argocd/monitoring.yaml --server-side
```

5. **Build image.** CI đẩy image lên ECR rồi ghi tag mới vào Git qua các pull request nhỏ do nó tự merge:

```
for w in go dotnet java node py; do gh workflow run $w-services-ci.yaml --ref main;
done
gh run list --limit 5            # chờ 5 run xanh hết
```

6. **Kiểm tra:** làm theo mục tiếp theo. Mọi thứ phải giống ảnh trong mục đó.

2.2 Kiểm tra hệ thống chạy tốt

Chạy 4 lệnh sau:

```
kubectl get nodes                       # 3 node, đều Ready
kubectl -n argocd get applications      # 14 app, đều Synced và Healthy
kubectl -n dev-eks get pods             # đều Running, số lần restart không tăng
kubectl -n dev-eks get hpa              # CPU của frontend dưới 70%
```

```
$ kubectl get nodes
NAME                                            STATUS   ROLES    AGE   VERSION
ip-10-18-2-103.ap-southeast-1.compute.internal   Ready    <none>   19m   v1.35.8-eks-3b4a6ca
ip-10-18-2-151.ap-southeast-1.compute.internal   Ready    <none>   19m   v1.35.8-eks-3b4a6ca
ip-10-18-3-199.ap-southeast-1.compute.internal   Ready    <none>   19m   v1.35.8-eks-3b4a6ca

$ kubectl -n dev-eks get pods
NAME                                     READY   STATUS    RESTARTS   AGE
adservice-dev-6f964d9445-7h6fp           1/1     Running   0          6m15s
cartservice-dev-659646cb65-wmx8c         1/1     Running   0          6m15s
checkoutservice-dev-856956f79f-qqm5v     1/1     Running   0          6m50s
currencyservice-dev-6fdb5dbc49-g26dr     1/1     Running   0          6m46s
emailservice-dev-799c485bc8-kjsnp        1/1     Running   0          6m55s
frontend-dev-59d4f86fc6-7hrfh            1/1     Running   0          6m11s
frontend-dev-59d4f86fc6-9ddkc            1/1     Running   0          5m46s
paymentservice-dev-75479bc798-w6rvc      1/1     Running   0          6m4s
productcatalogservice-dev-64c9b8d476-c2pfz   1/1     Running   0          6m4s
recommendationservice-dev-67c9ffc4c8-6zhs5   1/1     Running   0          6m46s
redis-cart-dev-65df6bd654-pxv74          1/1     Running   0          10m
shippingservice-dev-78b748699b-d2dfb     1/1     Running   0          6m47s

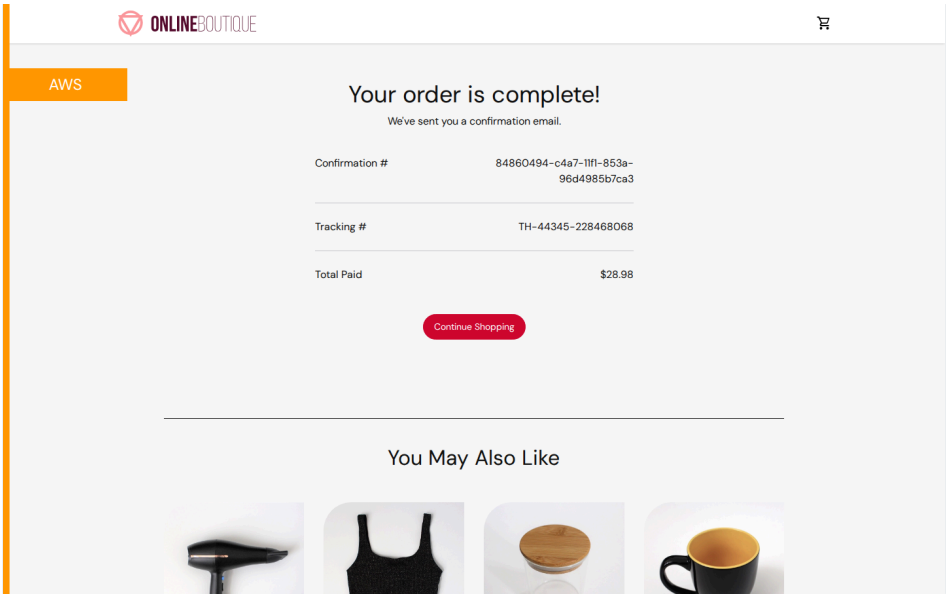
$ kubectl -n dev-eks get hpa,pdb
NAME                                                REFERENCE                  TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
horizontalpodautoscaler.autoscaling/frontend-dev    Deployment/frontend-dev    cpu: 1%/70%   2         4         2          10m

NAME                                    MIN AVAILABLE   MAX UNAVAILABLE   ALLOWED DISRUPTIONS   AGE
poddisruptionbudget.policy/frontend-dev   1               N/A               1                     10m

$ kubectl top nodes
NAME                                            CPU(cores)   CPU(%)   MEMORY(bytes)   MEMORY(%)
ip-10-18-2-103.ap-southeast-1.compute.internal   55m          2%       1278Mi          38%
ip-10-18-2-151.ap-southeast-1.compute.internal   78m          4%       1630Mi          49%
ip-10-18-3-199.ap-southeast-1.compute.internal   80m          4%       1440Mi          43%
```

Hình 1: Một cluster khỏe trông như thế này

Mở shop: lấy địa chỉ bằng `kubectl -n dev-eks get svc frontend-external-dev`, mở hostname ở cột `EXTERNAL-IP` trên trình duyệt rồi đặt thử một đơn.



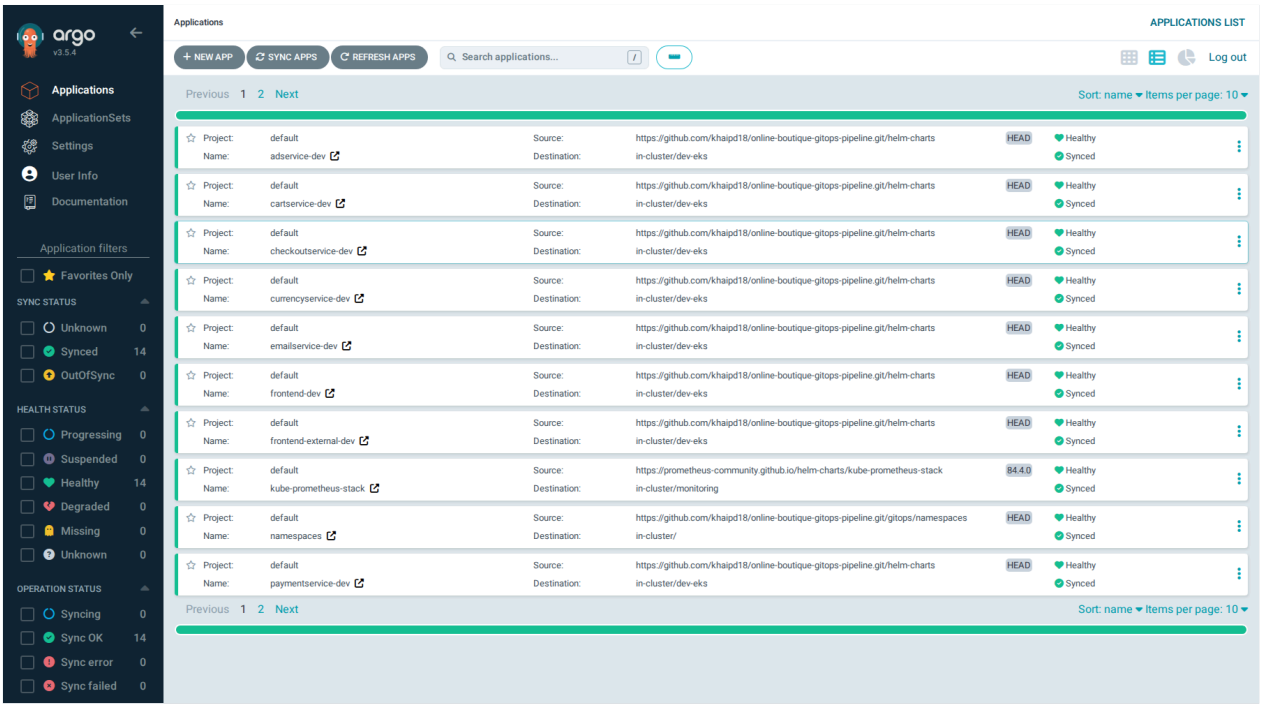
Hình 2: Đặt thử đơn hàng thành công: cả shop hoạt động

Mở các giao diện web (mỗi lệnh chạy liên tục; mở địa chỉ trên trình duyệt):

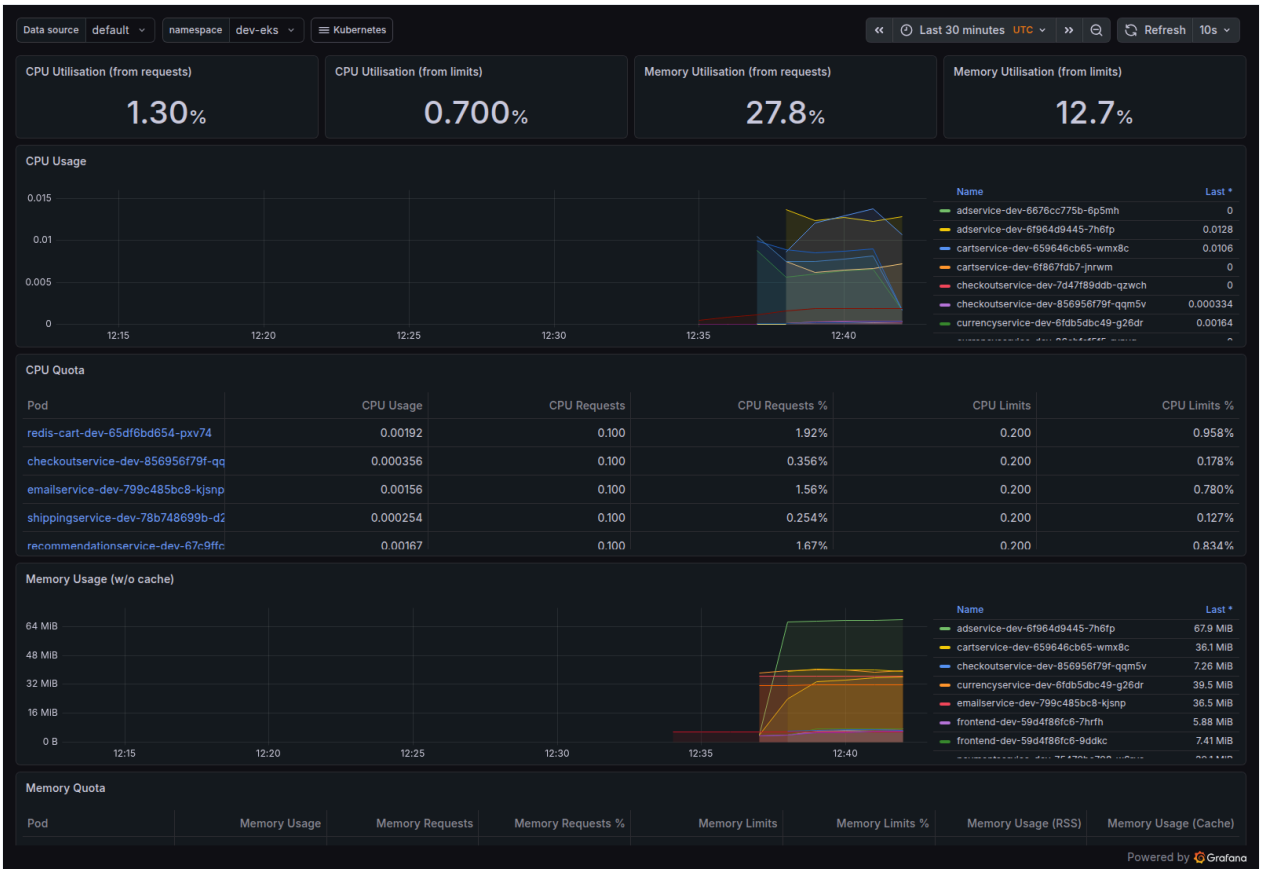
Giao diện	Lệnh	Địa chỉ
Argo CD	`kubectl -n argocd port-forward svc/argocd-server 8080:443`	https://localhost:8080
Grafana	`kubectl -n monitoring port-forward svc/kube-prometheus-stack-grafana 3000:80`	http://localhost:3000

User là `admin`. Lấy mật khẩu:

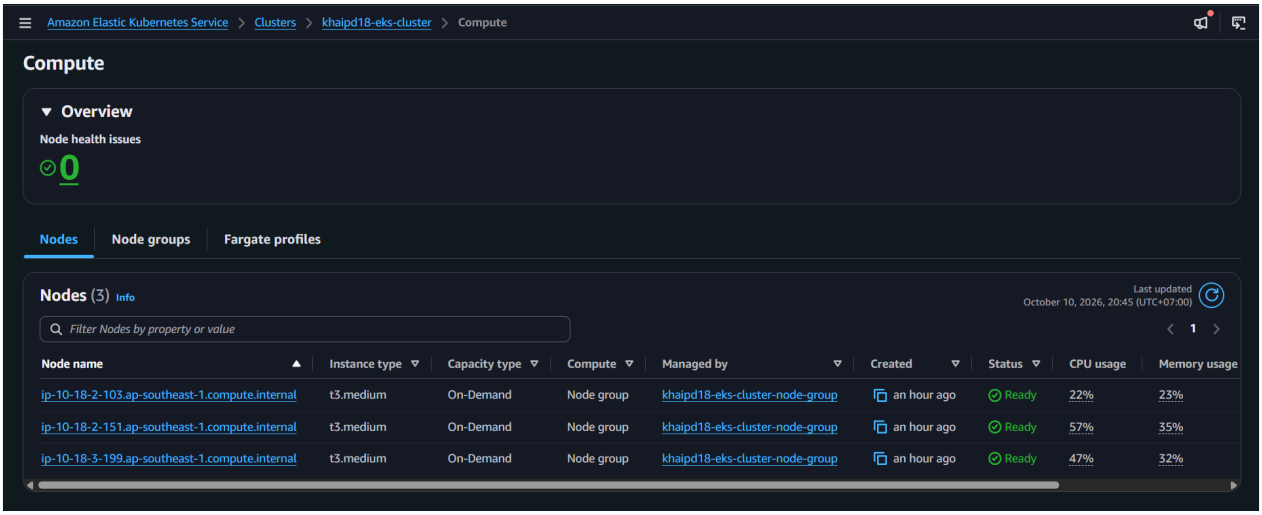
```
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath='{.data.password}'
| base64 -d; echo
kubectl -n monitoring get secret kube-prometheus-stack-grafana -o
jsonpath='{.data.admin-password}' | base64 -d; echo
```



Hình 3: Argo CD: mọi app đều Synced và Healthy



Hình 4: Grafana, dashboard “Kubernetes / Compute Resources / Namespace (Pods)”: CPU và memory của từng pod

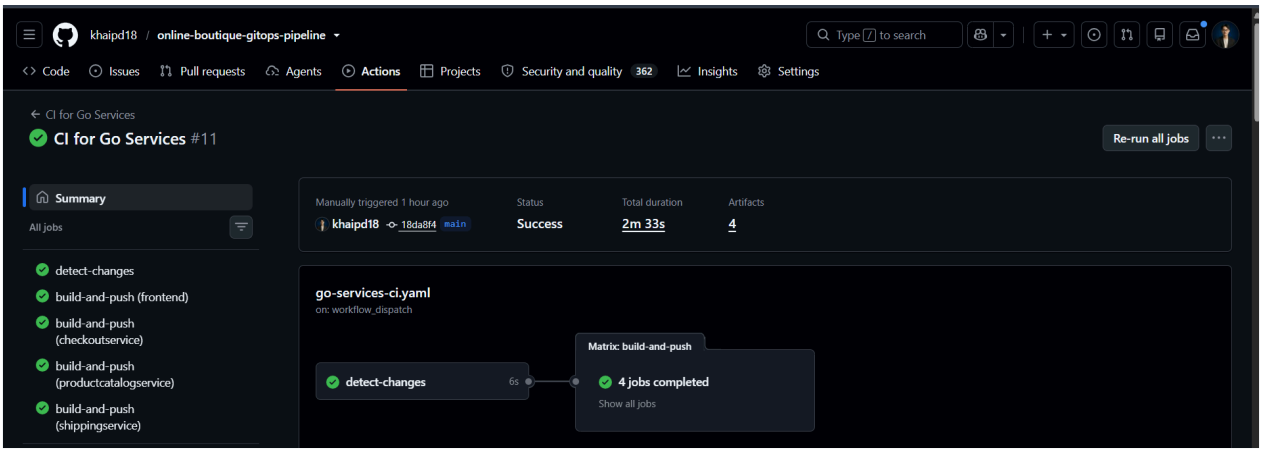


Hình 5: AWS console, EKS → Compute: ba node Ready

2.3 Ra bản mới cho một service

1. Merge thay đổi trong `src/<service>/` vào `main`.
2. GitHub Actions test, build và quét image, đẩy lên ECR rồi ghi tag mới vào `gitops/dev-eks/values-<service>.yaml` qua một pull request do nó tự merge.
3. Khoảng 3 phút sau Argo CD thấy tag mới và cập nhật pod.

Kiểm tra: `kubectl -n dev-eks rollout status deploy/<service>-dev`



Hình 6: Một run CI thành công trên GitHub Actions

2.4 Rollback một service

Mỗi tag image ứng với một commit, nên quay lại bản cũ chỉ là khôi phục tag trước đó trong Git:

```
git log --oneline -- gitops/dev-eks/values-<service>.yaml   # tìm commit đổi sang tag lỗi
git revert <commit>                                          # rồi push hoặc mở PR
```

> **CẢNH BÁO**
> Không dùng `kubectl set image` hay `kubectl rollout undo`. Argo CD sẽ đưa bản lỗi quay lại.

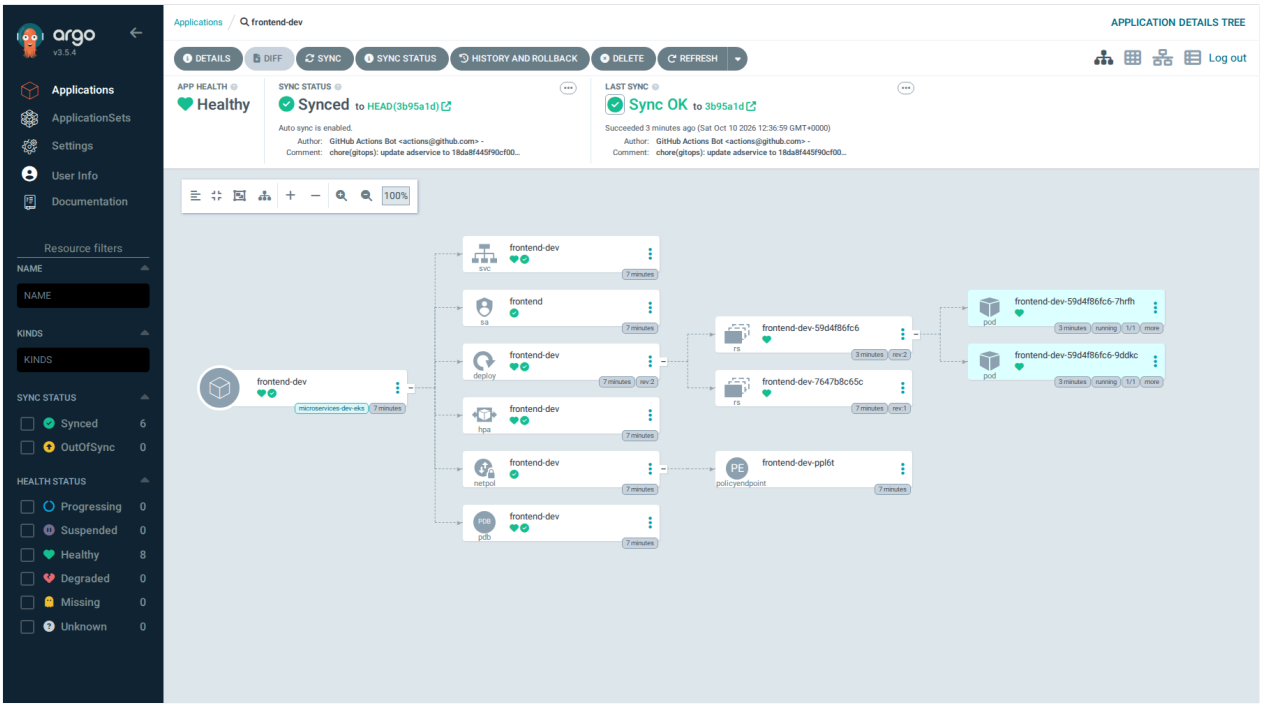
2.5 Đổi cấu hình hoặc scale một service

Mọi cấu hình của một service nằm trong `gitops/dev-eks/values-<service>.yaml`: biến môi trường, CPU và memory, số replica, autoscaling, và những service nào được gọi tới nó (`networkPolicy.allowFrom`).

1. Sửa file trên một nhánh.
2. Xem trước kết quả: `helm template <service>-dev helm-charts -f gitops/dev-eks/values-<service>.yaml`
3. Mở pull request rồi merge. Argo CD tự áp dụng.

> **MẸO**
> Khi một service bắt đầu gọi sang service khác, thêm nó vào `allowFrom` của service được gọi, nếu không lời gọi sẽ bị chặn.

Muốn đổi số **node**, sửa `eks_node_group_scaling_config` trong `terraform/variables.tf` rồi làm theo mục tiếp theo.



Hình 7: Argo CD hiển thị mọi thứ thuộc một service: Deployment, pod, HPA, NetworkPolicy, PodDisruptionBudget

2.6 Thay đổi hạ tầng

1. Sửa `terraform/` trên một nhánh rồi mở pull request.
2. GitHub chạy Checkov (kiểm tra bảo mật) và `terraform plan`. Đọc plan trong log của job.
3. Merge pull request.
4. Job **Terraform Apply** sẽ chờ. Trên GitHub: **Actions** → run đó → **Review deployments** → **production** → **Approve**.

> **QUAN TRỌNG**
> Chỉ duyệt khi đã đọc plan. Bước apply chạy với quyền admin đầy đủ.

2.7 Tắt môi trường

1. Xóa các app và load balancer trước (còn load balancer thì Terraform không xóa được VPC):

```
kubectl delete -f gitops/argocd/applicationset.yaml
kubectl -n dev-eks delete svc frontend-external-dev --ignore-not-found
```

2. Xóa hạ tầng (khoảng 10 phút):

```
cd terraform && terraform destroy
```

3. Báo CI rằng không còn tài khoản AWS: `gh variable delete AWS_ACCOUNT_ID`

Kiểm tra: `aws eks list-clusters --region ap-southeast-1` trả về danh sách rỗng. Bucket state và bảng khóa được giữ lại, gần như không tốn tiền.

3 Khi có sự cố

Tìm dấu hiệu trong bảng rồi làm theo cách sửa. Lệnh ở cột giữa thường cho biết nguyên nhân.

Dấu hiệu	Nguyên nhân thường gặp / cách xem	Cách sửa
CI lỗi ở bước "Configure AWS credentials"	Sai `AWS_ACCOUNT_ID`, hoặc chưa có IAM role. `gh variable list`	Sửa biến, hoặc tạo role bằng lần `terraform apply` đầu tiên
CI lỗi ở bước "Update GitOps through a pull request"	GitHub Actions chưa được phép tạo pull request trong repository	Settings → Actions → General → bật "Allow GitHub Actions to create and approve pull requests"
Terraform báo `Error acquiring the state lock`	Một lần chạy trước bị dừng giữa chừng. Chắc chắn không có gì đang chạy: `gh run list`	`terraform force-unlock <lock-id>`
Job Terraform Apply đứng ở **Waiting**	Đang chờ bạn duyệt	Duyệt trên GitHub (Thay đổi hạ tầng)
App trên Argo CD **OutOfSync** hoặc **Degraded**	Values sai hoặc pod không khởi động được. `kubectl -n argocd describe application <app>`	Sửa values trong Git
Pod bị `ImagePullBackOff`	Tag image không có trên ECR. `kubectl -n dev-eks describe pod <pod>`	Chạy CI của service, hoặc rollback
Pod bị `CrashLoopBackOff` / `OOMKilled`	Cấu hình sai hoặc thiếu memory. `kubectl -n dev-eks logs <pod> --previous`	Sửa values hoặc rollback
Pod báo `forbidden: violates PodSecurity`	Values đã bỏ các thiết lập bảo mật an toàn	Bỏ phần ghi đè đó trong file values
Lời gọi giữa các service bị timeout	Bên gọi chưa có trong `allowFrom`. `kubectl -n dev-eks get networkpolicy`	Thêm bên gọi vào `allowFrom`
Không mở được địa chỉ shop	Load balancer chưa sẵn sàng, hoặc `frontend` đang lỗi. `kubectl -n dev-eks get pods`	Chờ vài phút; sửa `frontend`
Pod nằm mãi ở `Pending`	Node không còn chỗ. `kubectl describe pod <pod>`	Giảm replica hoặc thêm node
HPA hiện `<unknown>`	Metrics Server chưa sẵn sàng. `kubectl top pods -n dev-eks`	Apply lại Terraform (nó cài Metrics Server)
`kubectl drain` không bao giờ xong	PodDisruptionBudget giữ lại một pod `frontend` (có chủ đích)	Thêm node để pod chuyển sang, rồi thử lại
Giỏ hàng bị trống	`redis-cart` khởi động lại; giỏ hàng không lưu xuống đĩa	Bình thường với bản demo này
EKS console báo "Data unavailable"	Role dùng trên console chưa có quyền trong cluster	Apply lại Terraform (nó cấp quyền đọc cho role console)

```
$ kubectl -n dev-eks run psa-test --image=busybox:1.37 --restart=Never -- sleep 60
Error from server (Forbidden): pods "psa-test" is forbidden: violates PodSecurity "restricted:latest": allowPrivilegeEscalation !=
false (container "psa-test" must set securityContext.allowPrivilegeEscalation=false), unrestricted capabilities (container "psa-
test" must set securityContext.capabilities.drop=["ALL"]), runAsNonRoot != true (pod or container "psa-test" must set
securityContext.runAsNonRoot=true), seccompProfile (pod or container "psa-test" must set securityContext.seccompProfile.type to
"RuntimeDefault" or "Localhost")

$ kubectl -n dev-eks exec np-test -- nc -zv -w 3 frontend-dev 80
frontend-dev (172.20.100.89:80) open

$ kubectl -n dev-eks exec np-test -- nc -zv -w 3 paymentservice-dev 50051
nc: paymentservice-dev (172.20.151.236:50051): Connection timed out
command terminated with exit code 1

$ kubectl -n dev-eks exec np-test -- nc -zv -w 3 redis-cart-dev 6379
nc: redis-cart-dev (172.20.141.193:6379): Connection timed out
command terminated with exit code 1
```

Hình 8: Bảo mật hoạt động đúng thiết kế: pod thiếu thiết lập bảo mật bị từ chối; pod không được phép thì không gọi được `paymentservice` hay `redis-cart`

QUAN TRỌNG

Nếu người dùng bị ảnh hưởng (shop sập, không checkout được), rollback trước rồi mới điều tra. Ghi lại sự việc vào một GitHub issue.

Phụ lục

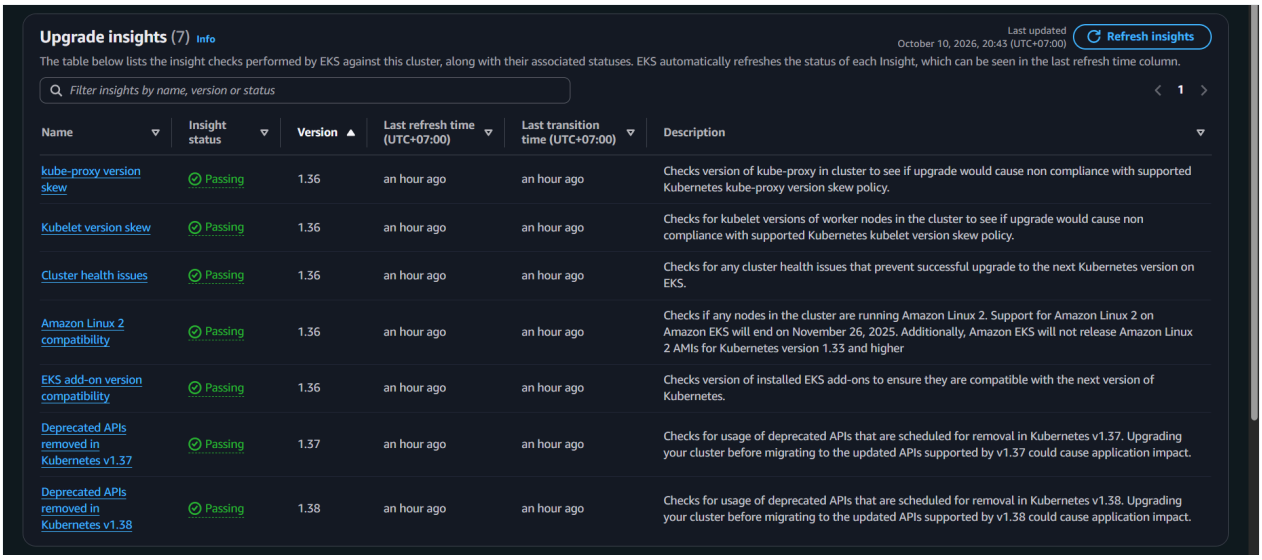
A. Nâng phiên bản Kubernetes

EKS hỗ trợ Kubernetes 1.35 tới ngày 27/03/2027 [AWS]. Mỗi lần chỉ nâng một phiên bản:

1. Xem **EKS console → Upgrade insights**: mọi mục phải **Passing**.
2. Đổi `eks_k8s_version` trong Terraform rồi apply. Bước này nâng control plane.
3. Nâng node; Terraform không tự làm việc này [AWS]:

```
aws eks update-nodegroup-version --cluster-name khaipd18-eks-cluster \
  --nodegroup-name khaipd18-eks-cluster-node-group --kubernetes-version <version>
```

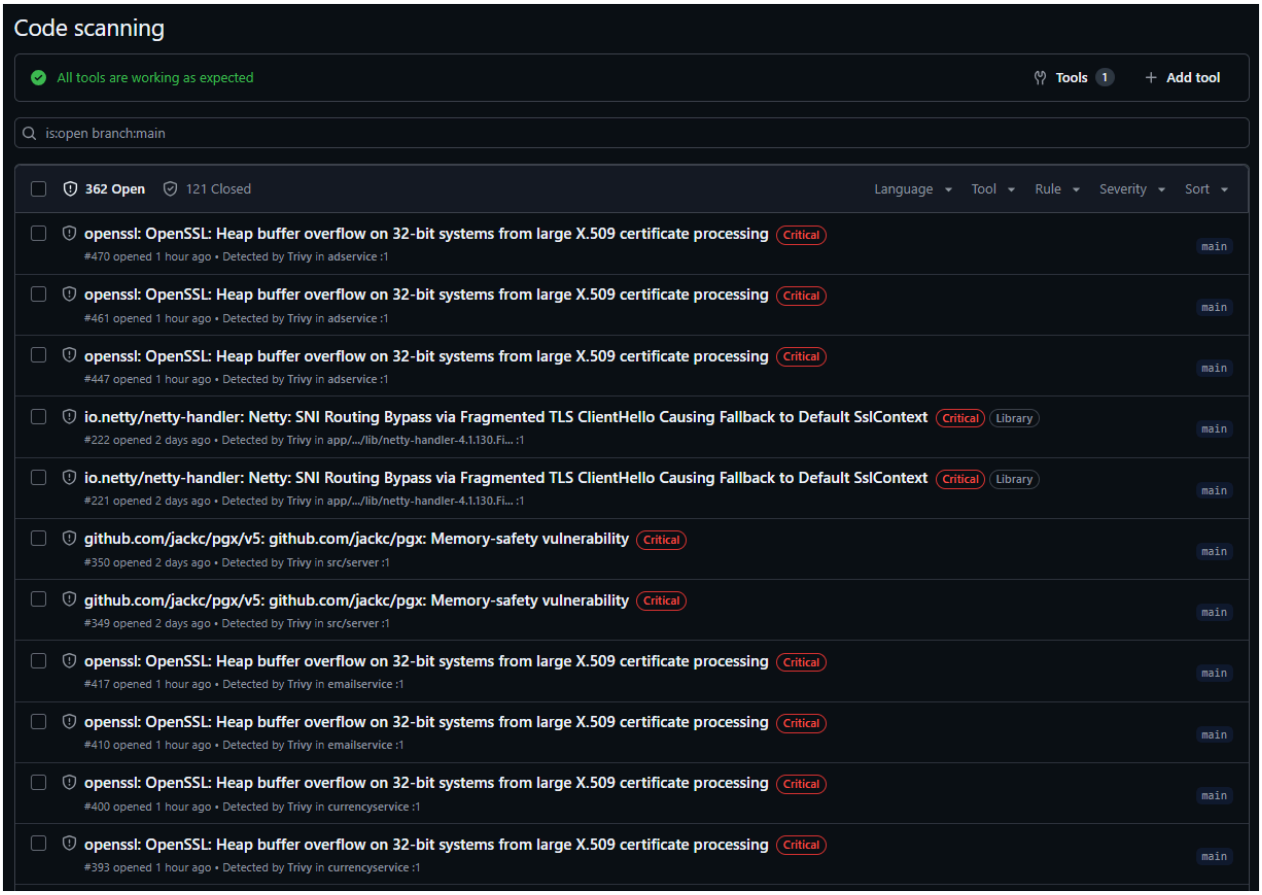
4. Nâng phiên bản các add-on (các biến `eks_*_version`). Xem phiên bản bằng `aws eks describe-addon-versions --addon-name <tên> --kubernetes-version <version>`.



Hình 9: EKS upgrade insights: mọi mục đều Passing

B. Lỗ hổng bảo mật

- **Checkov (Terraform):** sửa code nếu được. Nếu chấp nhận một finding, ghi rõ lý do rồi cập nhật baseline trong một pull request: `checkov --config-file .checkov.yaml -d terraform --framework terraform --create-baseline`
- **Trivy (image):** xem trên GitHub → **Security** → **Code scanning**. Các lỗ hổng đến từ code ứng dụng trong `src/`, phần repository này không sửa, nên chúng được báo cáo nhưng không chặn CI.



Hình 10: GitHub code scanning: finding của Trivy theo từng image

C. Chi phí

Khoảng **0,40 USD mỗi giờ** khi môi trường chạy, chưa tính load balancer và các khoản nhỏ (giá on-demand ở `ap-southeast-1`, lấy từ AWS Price List API): EKS control plane 0,10; ba node `t3.medium` 0,16; sáu ENI của interface endpoint 0,08; NAT Gateway 0,06. Xem chi phí thật trên AWS Cost Explorer. Không dùng thì tắt môi trường.

D. Khôi phục

Mất gì	Lấy lại thế nào
Toàn bộ cluster hoặc tài khoản	Dựng lại môi trường: mọi thứ được tạo lại từ Git trong khoảng 45 phút
Terraform state	Bucket state giữ các phiên bản cũ: khôi phục phiên bản trước của `dev/terraform.tfstate`
Một lần deploy lỗi	Rollback service
Giỏ hàng, dữ liệu lịch sử Grafana	Không được lưu; sẽ mất

E. Service và port

Service	Ngôn ngữ	Port	Nhận lời gọi từ	CPU req/limit	Memory req/limit
frontend	Go	80 → 8080	any (LoadBalancer)	100m / 200m	64Mi / 256Mi
checkoutservice	Go	5050	frontend	100m / 200m	64Mi / 128Mi
productcatalogservice	Go	3550	frontend, checkout, recommendation	100m / 200m	64Mi / 128Mi

shippingservice	Go	50051	frontend, checkout	100m / 200m	64Mi / 128Mi
cartservice	C#	7070	frontend, checkout	200m / 300m	64Mi / 128Mi
adservice	Java	9555	frontend	200m / 300m	180Mi / 300Mi
currencyservice	Node.js	7000	frontend, checkout	100m / 200m	64Mi / 128Mi
paymentservice	Node.js	50051	checkout	100m / 200m	64Mi / 128Mi
emailservice	Python	8080	checkout	100m / 200m	64Mi / 128Mi
recommendationservice	Python	8080	frontend	100m / 200m	250Mi / 500Mi
redis-cart	Redis	6379	cartservice	100m / 200m	64Mi / 128Mi

F. Phiên bản

Thành phần	Phiên bản	Khai báo ở
Terraform	1.14.8	terraform.yaml, README
AWS provider	6.39.0	terraform/versions.tf
Kubernetes (EKS)	1.35	terraform/variables.tf
VPC CNI add-on	v1.21.1-eksbuild.7	terraform/variables.tf
CoreDNS add-on	v1.13.2-eksbuild.4	terraform/variables.tf
kube-proxy add-on	v1.35.3-eksbuild.2	terraform/variables.tf
Metrics Server add-on	v0.9.0-eksbuild.11	terraform/variables.tf
kube-prometheus-stack	84.4.0	gitops/argocd/monitoring.yaml
Argo CD	stable manifest	README, step 3
Checkov	3.3.22	.github/workflows/*
Trivy	0.75.0	.github/actions/trivy-scan
Redis (cart store)	8.10.2	gitops/dev-eks/values-redis-cart.yaml